Q4(a): (2 points) If you are given the following sorted arrays:

```
A = [ 3,  7, 22, 29, 31, 53, 67 ]
B = [ 5, 11, 13, 17, 23, 37, 41, 43, 97 ]
```

What is the 7th smallest element in the *union* of the two arrays?

Answer (7th smallest): 22. [Xmas comes early again.]

(b) (8 points) [Finding *k*th smallest in two sorted arrays]
You are given two *sorted* arrays, $A[1..m]$ of size m and $B[1..n]$ of size n. You can assume that the two arrays are sorted in *increasing order* and that the elements are *all distinct* (there are *no repeated elements*). Give an $O(\lg m + \lg n)$ time algorithm for computing the kth smallest element in the *union* of the two arrays.

Discussion, Solution Sketch, and Feedback:

Before we do anything at all, let's do the following *puzzle*:

> Suppose $A[1..m]$ is a *sorted* array of family incomes of m people in Switzerland, and $B[1..n]$ is a *sorted* array of family incomes of n people in *poor country* XYZ.
>
> Now, we know median income of the m people in A (Switzerland) is $A[m/2]$.
> And, we know median income of the n people in B (poor XYZ) is $B[n/2]$.
> We want the *median* income of all the $(m+n)$ people in $A \cup B$.
>
> **Q1:** *Must* the median of the $(m+n)$ people in $A \cup B$ be either $A[m/2]$ or $B[n/2]$?
>
> **Q2:** If we want the kth smallest income of the $(m+n)$ people in $A \cup B$, and we have found i and j with $(i + j) = k$, must the answer *necessarily be* $A[i]$ or $B[j]$?

Both are FALSE, of course! (Hopefully, my use of *Switzerland* and *XYZ* helped.)

Yet, this error is, by far, ***the most common error*** for this problem.

Like students in the "*Cattywampus*" article, many of you *let your mind go to sleep*. You did not try to justify if this is *really true* before accepting it, or in this case, committing it to your "binary-search" code as *the* terminating condition, as in:

if $((i+ j) = k)$ **then return** $max\ \{A[i], B[j]\}$ (* found answer; *)

This is an ERROR. Why?
If $((i+ j) = k)$ and $(A[i] < B[j])$, then we know $B[j] \geq B[1..(j-1)]$ ($(j-1)$ elements) and also $B[j] \geq A[1..i]$ (i elements) – altogether $(i+j-1) = (k-1)$ elements.
But, we can only say that $B[j]$ is larger than at least $(k-1)$ elements (inclusive of $A[i]$), namely, $Rank(B[j]) \geq k$.

We cannot conclude yet if $Rank(B[j])=k$, since there could be more items in A (such as $A[i+1]$, $A[i+2]$, and so on) that are also less than $B[j]$. So, we cannot tell for sure if $B[j]$ is indeed the answer!!!

Remember this mathematician joke?
All we know is that there is *at least one horse* in Texas, and that *at least one side* of that one horse ***is black***.

> **Mathematician:** "No!
> All we know is that
> there is *at least one* horse in Texas, and that
> *at least one side* of *that one horse* is **black**."

So, please be alert, *don't let your mind go to sleep again*.

In the following pages, we shall work the solution from the simplest (but slow) to more sophisticated (and faster) ones.

Soln 1: (No brainer) Combine A & B into $C[1..(n+m)]$ sort it, and return $C[k]$.
Simple to understand, prove correctness. But *too slow*. $O((n+m)\lg(n+m))$.

Soln 2: (Refine from Soln 1; Arrays A and B are already sorted)
Merge A & B into $C[1..(n+m)]$ sorting it, and return $C[k]$.
Simple to understand, prove correctness. Faster but *still too slow*. $O(n+m)$.

To break the $O(n+m)$ barrier, we cannot sort the elements into $C = A \cup B$.
Instead, we work directly with array A & B. Lets' first see an example.

$A = [\ 2,\ 3,\ 6,\ 9,\ 18\ ]$
$B = [\ 8,\ 10,\ 12,\ 15,\ 20\ ]$

$C = [\ 2,\ 3,\ 6,\ 8,\ 9,\ 10,\ 12,\ 15,\ 18,\ 20\ ]$

If we look at the example above, can see that
$Rank(2) = 1$, $Rank(3) = 2$, $Rank(6) = 3$, $Rank(9) = 5$, $Rank(18) = 9$
$Rank(8) = 4$, $Rank(10) = 6$, $Rank(12) = 7$, $Rank(15) = 8$, $Rank(20) = 10$

Soln 3: Slow Sequential and Binary Search
To find rank of $A[x]$ (x=1,…,m), we can do binary search in array $B[1..n]$ to find the position y such that $B[y] \le A[x] \le B[y+1]$. Then we know $Rank(A[x]) = (x+y)$.
Then, we do the same and find $Rank(B[y])$, (y=1,…,n).
One of these *Ranks* must be k and that is the answer we need.

This algorithm is also easy, but it is slow, $O(m\lg n + n\lg m)$.

Soln 4: Better Binary Search with Binary Search
To improve this, don't have to use sequential search on $A[1..m]$ or $B[1..n]$.

We use binary search on $A[1..m]$ to find $A[x]$ with $Rank(A[x])$ closest to k.
This uses ($\lg m$) binary searches (each costing ($\lg n$) time). Total: ($\lg m$)($\lg n$)
We use binary search on $B[1..n]$ to find $B[x]$ with $Rank(B[y])$ closest to k.
This uses ($\lg n$) binary searches (each costing ($\lg m$) time). Total: ($\lg m$)($\lg n$)

One of these two will be of rank k, and is the answer.
This algorithm is faster, but still $O((\lg n)(\lg m))$.

Soln 5: (Elegant and fast solution)
To motivate our final solution, we add in one final idea: *Maintain invariance*.

Search $A[i]$ and $B[j]$, *while always maintaining the invariance* $(i + j) = k$.
(To maintain this invariance, for any given i, we always set $j = k - i$.)

Then, if $(A[i] = B[j])$, then $A[i]$ (or equivalently $B[j]$) is the rank k element.
(Note: For this quiz question, we have assumed that the elements are *all* distinct, and so this condition cannot occur.)

If $(A[i] < B[j])$, then $Rank(B[j]) \geq k$. From discussion on Soln 3,4, $Rank(B[j]) = k$ when $A[i] < B[j] < A[i+1]$. Else, $Rank(B[j]) > k$, and so we need to "increase i".

Equivalently, if $(A[i] > B[j])$, then $Rank(A[i]) \geq k$. And also $Rank(A[i]) = k$ when $B[j] < A[i] < B[j+1]$. Else, $Rank(A[i]) > k$, and so we need to "decrease i".

Next, we do some simple problem reductions:
The k smallest number in $A \cup B$ must be within the first k numbers in A or first k numbers in B (since $Rank(A[k+1]) \geq (k+1)$ and $Rank(B[k+1]) \geq (k+1)$.
So, we define $m' = min\ \{m, k\}$ and $n' = min\ \{n, k\}$.
We only need to consider the arrays $A[1..m']$ and $B[1..n']$.

We do binary search on $A[1..m']$ to find the correct value of i with $Rank(A[i])=k$ or j with $Rank(B[j]) = k$. In this binary search, we keep track of sub-range $A[la..ra]$ of array A. Initialize $la = 1$, $ra = m'$.

We define the mid-point, $i = (la+ra)/2$ in subarray $A[la..ra]$.
And to *maintain invariance*, we set $j = (k - i)$. The run the D&C method above.
Putting it all together, we have the following pseudo-code:

Pseudo-code for Soln 5:

```
Set m' = min {m, k};
Set la = 1;  ra = m';
While (la < ra) do begin
  Set i = (la+ra)/2;            (* mid-point in subarray A. *)
  Set j = (k – i);              (* maintain invariance *)

  If (A[i] = B[j])  then  return A[i], Stop;                (*Rank(A[i]) = k *)

  If (A[i] < B[j])  then                                    (*Rank(B[j]) ≥ k *)
     if (B[j] < A[i+1]) then return B[j], Stop;             (* A[i]<B[j]<A[i+1] *)
                        else set la = (i+1)   (* Rank(B[j]) > k, search right half of A *)

  else if (B[j] < A[i])  then                               (*Rank(A[i]) ≥ k *)
     if (A[i] < B[j+1]) then return A[i], Stop;             (* B[j] < A[i] < B[j+1]. *)
                        else set ra = (i–1)   (* Rank(A[i]) > k, search left half of A *)
endwhile
```

Complexity Analysis: Each time round the while loop, we do only $O(1)$ computations, and then *either* the algorithm terminates early, or else we search the *left half* or the *right half* of the sub-range of A. Thus, only $O(\lg m')$ iterations and the running time is $O(\lg m') = O(\lg n + \lg m) = O(max\ \{\lg n, \lg m\})$.

Let's try some examples with A and B, as given:

```
A = [ 3,  7, 22, 29, 31, 53, 67 ]
B = [ 5, 11, 13, 17, 23, 37, 41, 43, 97 ]
```

Example: Try k=7
$k = 7$, $m' = min\{7, 7\} = 7$; $n' = min\{9, 7\} = 7$;

Iteration 1: $la = 1$, $ra=7$, $i = 4 = (1+7)/2$; $j = 3 = (7–4)$;
Compare ($A[i]$ with $B[j]$): $A[4]=29$, $B[3]=13$. Case 3: ($B[3] < A[4]$);
Check $A[4] \le B[4]=17$. NO. So set $ra=3 = 4–1$ (* Search left half of A *)

Iteration 2: $la = 1$, $ra=3$, $i = 2 = (1+3)/2$; $j = 5 = (7–2)$;
Compare ($A[i]$ with $B[j]$): $A[2]=7$, $B[5]=23$. Case 2: ($A[2] < B[5]$);
Check $B[5] \le A[3]=22$. NO. So set $la=3 = 2+1$ (* Search right half of A *)

Iteration 3: $la = 3$, $ra=3$, $i = 3 = (3+3)/2$; $j = 4 = (7–3)$;
Compare ($A[i]$ with $B[j]$): $A[3]=22$, $B[4]=17$. Case 3: ($B[4] < A[3]$);
Check $A[3] \le B[5]=23$. YES. Return $A[3]=22$. ANSWER.

Another Example: Try k=6
$k = 6$, $m' = min\{7, 6\} = 6$; $n' = min\{9, 6\} = 6$;

Iteration 1: $la = 1$, $ra=6$, $i = 3 = (1+6)/2$; $j = 3 = (6–3)$;
Compare ($A[i]$ with $B[j]$): $A[3]=22$, $B[3]=13$. Case 3: ($B[3] < A[3]$);
Check $A[3] \le B[4]=17$. NO. So set $ra=2 = 3–1$ (* Search left half of A *)

Iteration 2: $la = 1$, $ra=2$, $i = 1 = (1+2)/2$; $j = 5 = (6–1)$;
Compare ($A[i]$ with $B[j]$): $A[1]=3$, $B[5]=23$. Case 2: ($A[1] < B[5]$);
Check $B[5] \le A[2]=7$. NO. So set $la=2 = 1+1$ (* Search right half of A *)

Iteration 3: $la = 2$, $ra=2$, $i = 2 = (2+2)/2$; $j = 4 = (6–2)$;
Compare ($A[i]$ with $B[j]$): $A[2]=7$, $B[4]=17$. Case 2: ($A[2] < B[4]$);
Check $B[4] \le A[3]=23$. YES. Return $B[4]=17$. ANSWER.

```
A = [ 3,  7, 22, 29, 31, 53, 67 ]
B = [ 5, 11, 13, 17, 23, 37, 41, 43, 97 ]
```

Another Example: Try k=4
$k = 4$, $m' = min\{7, 4\} = 4$; $n' = min\{9, 4\} = 4$;

Iteration 1: $la = 1$, $ra=4$, $i = 2 = (1+4)/2$; $j = 2 = (4–2)$;
Compare ($A[i]$ with $B[j]$): $A[2]=7$, $B[2]=11$. Case 2: ($A[2] < B[2]$);
Check $B[2] \le A[3]=22$. YES. Return $B[2]=11$. ANSWER.

NOTE: Actually, if $m \ge k$ and $n \ge k$.
Then we focus only on $A[1..k]$ and $B[1..k]$ (both sorted), and ask for the median.

There is a *different elegant solution* to this special case, takes only $O(\lg k)$ time. I told Steven about it and he posted on Facebook right after the Mid-Term.

Common Errors (for Q4(b)):

* The above-mentioned error (from puzzle) and *many variants* of the same error.

* Many of you made me "PLAY COMPILER".
  You gave only code or pseudo-code without any *explanation or justification*.
  So, you made me "go into your brain" to decode what you are doing.

  If you do this in the final exam, good luck to you.
  (I can't figure out your code and there's no explanation or justification,
  I assume it is wrong or it is incomprehensible. Either will get low marks.)

* Some of you only manipulate the indices of A and B (checking, updating etc),
  but without comparing $A[i]$ or $B[j]$. (Check your code)

* Don't just say binary search. Binary search on what? (There are two arrays!)
  Better to say Binary search for x in sorted array $B[k..h]$.

* You "track" your binary search with only 1 parameter (the mid-point).
  No way to do this, unless you also track the "step-size".
  If that is the case, you might as well track the start and end of the subrange.
  eg: A[$la..ra$], track with la and ra.